



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 03

Team No.: 14

Project Title: Linux Inter-Process Communication Using Pipes and FIFOs

Team Members

Roll Number	Student Name	Signature
2520030471	SAI ANANYA ROY	
2520030504	DEVUNI SHREENITHA	

1. Abstract

Linux Inter-Process Communication Using Pipes and FIFOs is a system programming project that demonstrates how separate processes can communicate and exchange data in a Linux environment. Since processes normally have independent memory spaces, a suitable communication mechanism is required for them to share information and coordinate their activities. The main objective of this project is to understand and implement IPC using unnamed pipes and named pipes (FIFOs) while demonstrating process creation, communication, synchronization, and resource management. The proposed approach begins by creating processes using `fork()`. An unnamed pipe is then used to establish communication between related processes, such as a parent and child, where one process writes data and the other reads it. For communication between independent processes, a FIFO is created using `mkfifo()`, allowing processes to exchange data through a named communication channel. The project will be implemented in C on Linux/Ubuntu using Linux/POSIX system-programming interfaces. Important system calls and functions include `fork()`, `pipe()`, `mkfifo()`, `open()`, `read()`, `write()`, `close()`, and `wait()`. Proper synchronization and error handling will be included to

ensure reliable communication and safe resource usage. Different test cases will be performed by changing the number of processes and the messages exchanged between them. The expected outcome is a working Linux-based application that clearly demonstrates communication between related and independent processes. The project will provide practical understanding of process creation, IPC, pipes, FIFOs, synchronization, system calls, and Linux/POSIX programming, with results verified through terminal outputs and test cases.

2. Problem Statement

In a multitasking Linux operating system, multiple processes often need to exchange data and coordinate their activities. Since processes normally have separate memory spaces, directly sharing data between them is not simple. This project addresses this problem by implementing *Inter-Process Communication (IPC) using pipes and FIFOs, allowing processes to send and receive data in a controlled and reliable manner.

3. Objectives

- To understand Inter-Process Communication (IPC) in Linux.
- To implement communication using pipes and FIFOs.
- To use Linux/POSIX system calls such as `fork()`, `pipe()`, `mkfifo()`, `read()`, and `write()`.
- To demonstrate process creation, communication, and synchronization.
- To test and verify communication between processes using Linux terminal outputs.

4. Proposed Methodology

The project will first create processes using `fork()`. For communication between related processes, an unnamed pipe will be created, where one process writes data and another process reads it. For communication between independent processes, a FIFO (named pipe) will be created so that separate processes can exchange data through a common communication channel. The program will provide simple demonstrations of sending, receiving, and processing messages while handling synchronization and proper opening and closing of communication channels.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
--------------------------------------	------------------------

fork()	Process Creation
pipe()	Unnamed Pipe
mkfifo()	Named Pipe/FIFO
Named Pipe/FIFO	Data Transfer
open() and close()	File Descriptors/Resource Management
wait()	Process Synchronization

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030504	Devuni Shreenitha	Process creation using <code>fork()</code> , unnamed pipe implementation using <code>,</code> data transfer using <code>read()/write()</code> , synchronization using <code>wait()</code> , testing and documentation.
2520030471	Sai Ananya Roy	FIFO creation using <code>mkfifo()</code> , inter-process communication using <code>open()/read()/write()</code> , error handling, resource management, integration, testing and documentation.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and execution environment
C / C++	Main programming language
VS Code	Writing and managing source code

Linux Terminal	Running and testing the IPC programs
Linux System Calls	Implementing <code>fork()</code> , <code>pipe()</code> , <code>mkfifo()</code> , <code>read()</code> , <code>write()</code> , <code>open()</code> , <code>close()</code> , and <code>wait()</code>
Makefile	Compiling and building the complete project easily
Git & GitHub	Version control and storing the project code

8. Expected Outcome

The expected outcome is a working Linux-based application that successfully demonstrates inter-process communication using both pipes and FIFOs. The project will show how related and independent processes can exchange information, synchronize their activities, and safely handle communication channels. Through this implementation, the project will provide practical understanding of process creation, IPC, system calls, synchronization, and Linux/POSIX programming concepts. The results will be tested using different communication scenarios and verified through terminal outputs.

9. GitHub Repository

GitHub Repository Link: [Github](#)

Repository Created: _____

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____